MODULE 1 · LECTURE 22

Heuristic Search Techniques: Best-first Search and the A* Algorithm

How a heuristic estimate of the distance to the goal guides search, how greedy best-first search and A use it, why admissible and consistent heuristics make A* optimal, how heuristics are compared — traced by hand on a farm-to-mandi road map and run on a field-robot problem.*

Learning objectives

After this lecture you should be able to:

1. Explain what a heuristic function is and how informed search differs from uninformed search.
2. Describe best-first search and its evaluation function f(n).
3. Trace greedy best-first search and A* search by hand on a weighted road map.
4. Define admissible and consistent heuristics and explain why they make A* optimal.
5. Compare heuristics using dominance and the effective branching factor.
6. Outline faster and memory-saving variants and apply heuristic search to agricultural problems.

Scope: Lecture 21 covered **uninformed** search — breadth-first, depth-first, iterative deepening and uniform-cost search — which uses no knowledge of where the goal lies. This lecture adds that knowledge in the form of a **heuristic**. Robot navigation and field operations that rely on these methods are taken up in Lectures 24 and 27.

1. From blind search to informed search

A **heuristic** is problem-specific knowledge that helps a search decide which state looks most promising (Pearl, 1984). In search algorithms it takes the form of a **heuristic function h(n)**: an estimate of the cost of the cheapest path from node n to a goal, with h(goal) = 0. A heuristic is cheap to compute but need not be exact.

Problem	A simple heuristic h(n)
Road route from a farm to a mandi	Straight-line distance from the current place to the mandi, from GPS coordinates
Robot moving up, down, left or right across a field grid	Manhattan distance: |row difference| + |column difference| to the goal cell
8-puzzle (sliding tiles)	Number of misplaced tiles, or the sum of each tile's Manhattan distance from its home square

Best-first search is the general method: keep the frontier in a **priority queue** and always expand the node with the lowest value of an **evaluation function f(n)** (Dechter & Pearl, 1985). The choice of f(n) gives the different algorithms. Two quantities are used: **g(n)**, the cost of the path actually travelled from the start to n, and **h(n)**, the estimated cost from n to the goal.

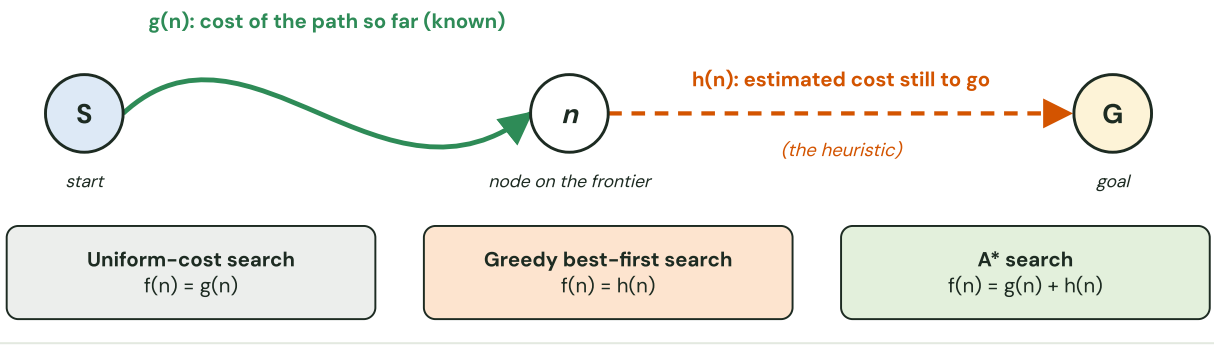
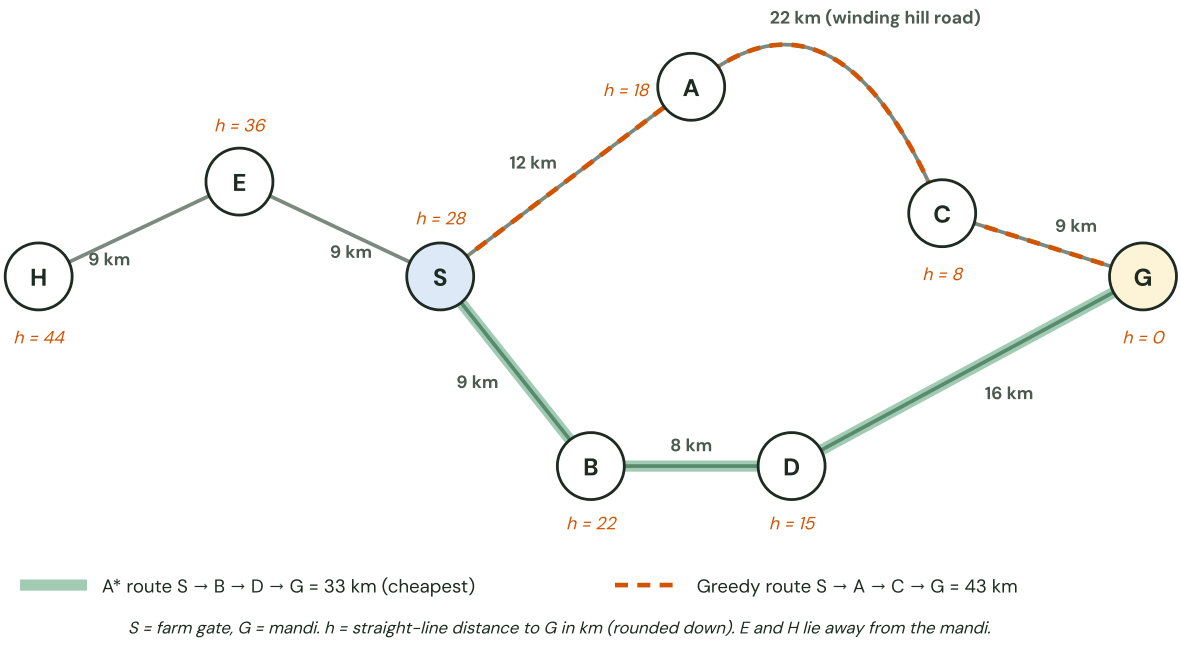


Figure 1. The two parts of an evaluation function and the algorithms they produce. Original diagram.

Uniform-cost: $f(n) = g(n)$ Greedy best-first: $f(n) = h(n)$ A*: $f(n) = g(n) + h(n)$

2. Greedy best-first search

Greedy best-first search uses **f(n) = h(n)**: it always expands the node that appears closest to the goal and ignores the cost already spent getting there (Russell & Norvig, 2020).



S = farm gate, G = mandi. h = straight-line distance to G in km (rounded down). E and H lie away from the mandi.

Figure 2. Roads from a farm gate S to a mandi G, with road lengths and straight-line distances h to the mandi. The road A–C is long because it winds over a hill. Original diagram; map and distances are illustrative.

Worked example 1 — greedy best-first search from the farm to the mandi

Step	Node expanded	Children added (h)	Frontier after the step, lowest h first
1	S (h = 28)	A (18), B (22), E (36)	A 18, B 22, E 36
2	A (h = 18)	C (8)	C 8, B 22, E 36
3	C (h = 8)	G (0)	G 0, B 22, E 36
4	G — goal	—	Route S → A → C → G = 12 + 22 + 9 = **43 km**

Greedy search reached the mandi after expanding only four nodes, but it chose village A because A *looks* close, without noticing the long hill road beyond it. The cheapest route, S → B → D → G, is 33 km.

Properties of greedy best-first search

- **Not optimal**: it ignores g(n), so it can return a longer route, as above.
- **Not complete** as tree search — it can oscillate between two states that both look close to the goal. With an explored set in a finite state space it does find a solution.
- **Time and space O(b^m)** in the worst case, where m is the maximum depth; a good heuristic usually reduces this greatly (Russell & Norvig, 2020).

3. The A* algorithm

A* (read "A-star") was introduced by Hart, Nilsson and Raphael (1968). It adds the cost already travelled to the estimate of the cost still to go:

$$f(n) = g(n) + h(n) \qquad = \text{estimated cost of the cheapest solution that passes through } n$$

A* therefore prefers nodes that are both cheap to reach and close to the goal. It uses two lists: **OPEN**, the frontier, sorted by f, and **CLOSED**, the nodes already expanded.

1. Put the start node S on OPEN with g(S) = 0 and f(S) = h(S). CLOSED is empty.
2. If OPEN is empty, report failure.
3. Remove from OPEN the node n with the smallest f(n) and put it on CLOSED.
4. If n is a goal, return the path to it by following the parent links back to S.
5. For each successor m of n, compute g = g(n) + c(n, m). If m is new, or this g is lower than the g recorded for m, set g(m) = g, f(m) = g(m) + h(m) and parent(m) = n, and put m on OPEN.
6. Go to step 2.

The goal test is made when a node is removed from OPEN, not when it is first generated. A goal generated early may still be reached later by a cheaper path, so stopping at generation could return a costlier route.

Worked example 2 — A* from the farm to the mandi

Step	Node expanded (f = g + h)	Successors added (f = g + h)	OPEN after the step, lowest f first
1	S: 0 + 28 = 28	A: 12 + 18 = 30; B: 9 + 22 = 31; E: 9 + 36 = 45	A 30, B 31, E 45
2	A: 30	C: 34 + 8 = 42	B 31, C 42, E 45
3	B: 31	D: 17 + 15 = 32	D 32, C 42, E 45
4	D: 32	G: 33 + 0 = 33	G 33, C 42, E 45
5	G: 33 — goal	—	Route S → B → D → G = 9 + 8 + 16 = **33 km**

Neighbours already on CLOSED (such as S) are ignored. A* first tried A, but once the 22 km hill road made C cost f = 42, the route through B and D (f = 32, then 33) became cheaper. A* expanded five nodes and found the cheapest route. Uniform-cost search, which uses no heuristic, also finds the 33 km route but expands seven nodes — including E and H, which lie away from the mandi. Their f values under A* (45 or more) exceed the cost of the solution, so A* never expands them.

4. Admissible and consistent heuristics

Let h*(n) be the true cost of the cheapest path from n to the goal. A heuristic is **admissible** if it never overestimates that cost:

$$\text{Admissible: } h(n) \le h^*(n) \text{ for every node } n \qquad \text{Consistent: } h(n) \le c(n, n') + h(n') \text{ for every road } n \to n'$$

Straight-line distance is admissible for road routes because no road between two places can be shorter than the straight line between them.

Node	S	A	B	C	D	E	H	G
h(n), straight line (km)	28	18	22	8	15	36	44	0
h*(n), cheapest road distance (km)	33	31	24	9	16	42	51	0

Every h value is at most the true cost, so the heuristic is admissible. It is also **consistent** (monotone): for every road, the estimate from one end is no more than the road length plus the estimate from the other end — the triangle inequality. For road A–C: 18 ≤ 22 + 8. Every consistent heuristic is also admissible.

Why an admissible heuristic makes A* optimal

Hart et al. (1968) proved that A* returns a minimum-cost path whenever h never overestimates. The argument in brief: suppose the cheapest route costs C*, and A* is about to stop at a goal reached by a costlier route, with f = g > C*. Some node n on the cheapest route is still on OPEN, and because h is admissible, f(n) = g(n) + h(n) ≤ C*. That node has a lower f, so A* would expand it first — the costlier goal cannot be chosen first. With a consistent heuristic, f never decreases along a path, so A* reaches every node by its cheapest route the first time it expands it, and closed nodes never have to be reopened (Russell & Norvig, 2020). For a consistent heuristic, no other algorithm that is guaranteed to find the cheapest path and uses the same heuristic must expand fewer nodes, apart from ties (Dechter & Pearl, 1985).

> **Worked example 3 — what if the heuristic overestimates?**
> Suppose h(B) is wrongly set to 40, although the true cost from B is only 24.
> Step 1: expand S → A (f = 30), B (f = 9 + 40 = 49), E (f = 45).
> Step 2: expand A → C (f = 42). Step 3: expand C → G (f = 43 + 0 = 43).
> Step 4: G has the lowest f (43 < 45 < 49) and is selected: A* returns S → A → C → G = **43 km**.
> Because B was overestimated, A* never looked at the cheaper 33 km route. Admissibility is what guarantees optimality.

5. How good is a heuristic?

- **h(n) = 0 for every node**: A* becomes uniform-cost search — safe, but no guidance.
- **h(n) = h*(n)**, a perfect estimate: A* goes straight along the cheapest path, apart from ties.
- Between the two, the closer an admissible heuristic is to the true cost, the fewer nodes A* expands.

If two admissible heuristics satisfy $h_2(n) \ge h_1(n)$ for every node, h_2 **dominates** h_1, and A* with h_2 will not expand more nodes than with h_1, apart from ties (Russell & Norvig, 2020). A common way to measure this is the **effective branching factor b***: if A* expands N nodes to find a solution at depth d, b* is the branching factor that a uniform tree of depth d would need to hold N + 1 nodes. A value close to 1 means the search went almost straight to the goal.

$$N + 1 = 1 + b^* + (b^*)^2 + \ldots + (b^*)^d$$

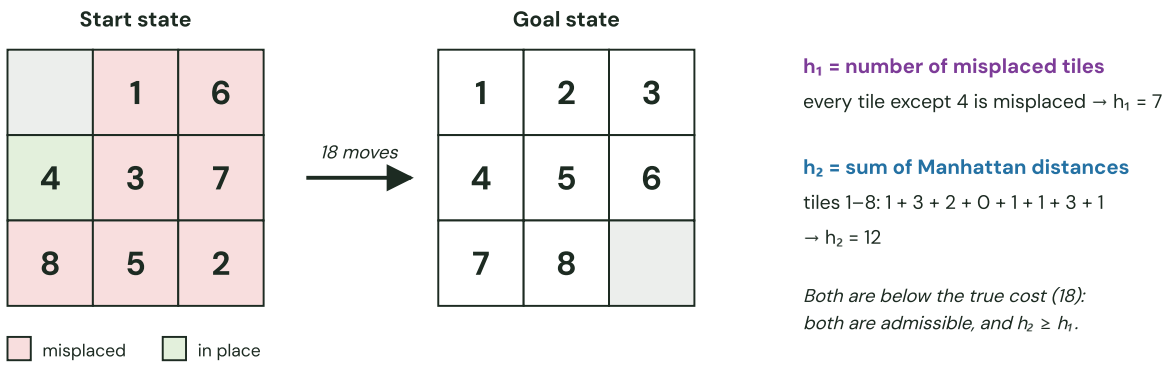


Figure 4. An 8-puzzle start state, its goal and two admissible heuristics. Original example.

Worked example 4 — two heuristics for the 8-puzzle

In the 8-puzzle, a tile next to the blank can slide into it; every move costs 1. For the start state in Figure 4, the misplaced-tile heuristic gives $h_1 = 7$ and the Manhattan-distance heuristic gives $h_2 = 12$. Both are admissible: every misplaced tile needs at least one move, and at least as many moves as its Manhattan distance. The shortest solution needs 18 moves.

Heuristic used by A*	Value at the start	Nodes expanded	Effective branching factor b*
None, h = 0 (uniform-cost search)	0	23,142	1.66
h_1 — misplaced tiles	7	1,372	1.39
h_2 — Manhattan distance	12	244	1.24

All three searches found the 18-move solution. The counts come from an A program run for this lecture, with ties broken in order of generation; other implementations give slightly different counts, but the pattern is the same: the dominant heuristic h_2 expands fewer than one-fifth as many nodes as h_1 and about one-hundredth as many as uniform-cost search.*

Inventing heuristics: relaxed problems

Good admissible heuristics often come from a **relaxed problem** — the same problem with some rules removed. If a tile could slide onto any neighbouring square, occupied or not, the exact cost would be h_2; if a vehicle could travel in a straight line instead of by road, the exact cost would be the straight-line distance. A solution to the relaxed problem can never cost more than one to the real problem, so its cost is an admissible heuristic (Russell & Norvig, 2020).

6. Variants and other heuristic search techniques

Technique	Idea	Trade-off
Weighted A*	f(n) = g(n) + w·h(n) with w > 1, giving the heuristic more weight (Pohl, 1970)	Expands fewer nodes; the route costs at most w times the optimum (Russell & Norvig, 2020)
Iterative-deepening A* (IDA*)	Depth-first search with a limit on f = g + h, raised step by step (Korf, 1985)	Optimal with an admissible h and needs memory only for the current path; repeats some work
Beam search	Keeps only the best k nodes on the frontier	Fast, small memory; may miss the solution or the best route
Hill climbing	Moves from the current state to the best neighbour and keeps no frontier	Tiny memory, but gets stuck at local maxima, plateaus and ridges; random restarts help (Russell & Norvig, 2020)
AO*	Heuristic search of AND–OR graphs, where a problem splits into sub-problems that must all be solved (problem reduction)	Finds the cheapest solution graph rather than a single path (Pearl, 1984)

Comparison

Strategy	f(n)	Complete?	Optimal?	Memory
Uniform-cost	g(n)	Yes, if every step costs more than zero	Yes	All generated nodes
Greedy best-first	h(n)	Only with an explored set in a finite space	No	All generated nodes; O(b^m) worst case
A*	g(n) + h(n)	Yes, if b is finite and step costs are above zero	Yes, if h is admissible (consistent for graph search)	All generated nodes — its main drawback
Weighted A*	g(n) + w·h(n)	Yes, under the same conditions as A*	No — cost at most w × optimal	Usually fewer nodes than A*
IDA*	g(n) + h(n) as a limit	Yes, under the same conditions as A*	Yes, if h is admissible	Linear, O(b·d)
Hill climbing	h(n) of neighbours	No	No	Only the current state

7. Worked example: a field robot crossing mud

A field robot must travel from S to a sampling point G on a 7 × 7 grid. Entering a normal cell costs 1; entering a muddy cell costs 3, because the robot slows down and uses more energy. The heuristic is the Manhattan distance to G. It is admissible and consistent: every move changes the row or column by one and costs at least 1.

Breadth-first, uniform-cost, greedy best-first and A* search for a field robot in Python (Lecture 8)

```
import heapq

FIELD = ["S . . . . . .",     # S = robot start, G = goal, # = obstacle,
         ". . ~ ~ ~ . .",     # ~ = muddy cell (costs 3 to enter; other cells cost 1)
         ". # ~ ~ ~ # .",
         ". # ~ ~ ~ # .",
         ". # . . . # .",
         ". . . # . . .",
         ". . . # . . G"]
grid = [row.split() for row in FIELD]
ROWS, COLS = len(grid), len(grid[0])
start, goal = (0, 0), (ROWS - 1, COLS - 1)

def neighbours(cell):                          # up, down, left, right
    r, c = cell
    for dr, dc in ((-1, 0), (1, 0), (0, -1), (0, 1)):
        nr, nc = r + dr, c + dc
        if 0 <= nr < ROWS and 0 <= nc < COLS and grid[nr][nc] != "#":
            yield (nr, nc)

def step_cost(cell):
    return 3 if grid[cell[0]][cell[1]] == "~" else 1

def h(cell):                                   # Manhattan distance to the goal
    return abs(cell[0] - goal[0]) + abs(cell[1] - goal[1])

def priority(kind, g, moves, cell):
    return {"Breadth-first": moves, "Uniform-cost": g,
            "Greedy best-first": h(cell), "A*": g + h(cell)}[kind]

def search(kind):
    count = 0                                  # tie-breaker: earlier entries first
    frontier = [(0, count, start, 0, [start])] # (priority, tie, cell, g, path)
    best_g, expanded = {start: 0}, set()
    while frontier:
        _, _, cell, g, path = heapq.heappop(frontier)
        if cell in expanded:
            continue
        expanded.add(cell)
        if cell == goal:
            return path, g, len(expanded)
        for nxt in neighbours(cell):
            g2 = g + step_cost(nxt)
            if nxt in expanded or g2 >= best_g.get(nxt, float("inf")):
                continue
            best_g[nxt] = g2
            count += 1
            heapq.heappush(frontier, (priority(kind, g2, len(path), nxt),
                                      count, nxt, g2, path + [nxt]))

for kind in ("Breadth-first", "Uniform-cost", "Greedy best-first", "A*"):
    path, cost, n = search(kind)
    print(f"{kind:18s} moves {len(path) - 1:2d}  cost {cost:2d}  cells expanded {n:2d}")
path, cost, n = search("A*")
show = [row[:] for row in grid]
for r, c in path[1:-1]:
    show[r][c] = "*"
print("A* route:")
print("\n".join("   " + " ".join(row) for row in show))

Output:
```

```
Breadth-first      moves 12  cost 18  cells expanded 41
Uniform-cost       moves 12  cost 12  cells expanded 39
Greedy best-first  moves 16  cost 16  cells expanded 18
A*                 moves 12  cost 12  cells expanded 25
A* route:
   S * * * * * .
   . . ~ ~ ~ * *
   . # ~ ~ ~ # *
   . # ~ ~ ~ # *
   . # . . . # *
   . . . # . . *
   . . . # . . G
```

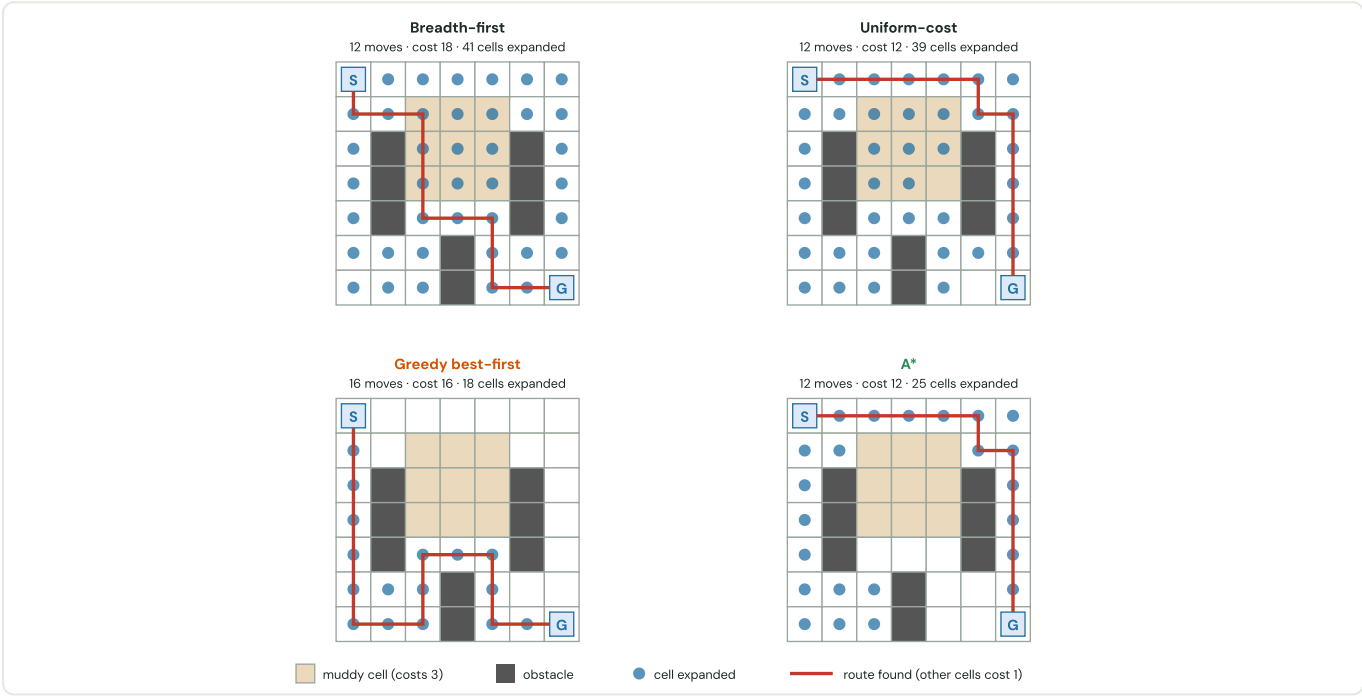


Figure 3. Cells expanded (blue dots) and routes found (red) by four strategies on the same field. Original diagram generated from the program's rules.

- **Breadth-first** found a route with the fewest moves (12), but it crosses the mud, so it costs 18.
- **Uniform-cost** found the cheapest route (cost 12) but expanded 39 cells, spreading out in every direction.
- **Greedy best-first** expanded only 18 cells but took a 16-move detour around the obstacles.
- **A*** found the cheapest route (cost 12) and expanded 25 cells — about 36% fewer than uniform-cost search.

The program was run and the output shown is its actual output. Changing the A priority to g + 2h (weighted A*, w = 2) and running it again gave a route of cost 14 after expanding 18 cells — faster, and within the guaranteed limit of 2 × 12 = 24.*

8. Uses in agriculture

Problem	How heuristic search is used
Point-to-point navigation of field robots and autonomous tractors	Path planners search a grid or map of the field for a short, obstacle-free route; path planning has been applied to agricultural ground robots for both field coverage and point-to-point navigation (Santos et al., 2020) (Lectures 24 and 27)
Farm-to-market and input-delivery routing	Finding the shortest road route from a farm to a mandi, cold store or procurement centre, with straight-line distance from GPS coordinates as an admissible heuristic (Lecture 14)
Drone flight planning	Planning routes between spraying or imaging waypoints that avoid trees, power lines and no-fly zones (Lectures 15 and 25)
Scheduling field operations	Searching sequences of irrigation, spraying or harvesting tasks, with a heuristic estimate of the remaining time or cost to prune poor sequences
Diagnosis in expert systems	Ordering which disease or deficiency hypotheses to test first, using heuristic scores of likelihood (Lecture 17)

9. Key takeaways

- A heuristic h(n) estimates the cost from n to the goal; informed search uses it to choose which node to expand.
- Best-first search expands the frontier node with the lowest f(n): uniform-cost uses g, greedy uses h, A* uses g + h.
- Greedy best-first search is fast but not optimal: on the farm-to-mandi map it chose a 43 km route instead of the 33 km one.
- A* (Hart, Nilsson & Raphael, 1968) is complete and optimal when h is admissible; a consistent h also avoids reopening nodes.
- An overestimating heuristic can make A* miss the cheapest route.
- Better (dominant) admissible heuristics expand fewer nodes: 23,142 → 1,372 → 244 in the 8-puzzle example.
- A* stores every generated node; IDA*, weighted A*, beam search and hill climbing trade guarantees for speed or memory.
- In agriculture, heuristic search plans robot, tractor and drone routes, farm-to-market trips and task schedules.

10. Examination questions

Short-answer questions (2 marks)

Q1. What is a heuristic function?

An estimate h(n) of the cost of the cheapest path from node n to a goal, with h(goal) = 0.

Q2. Distinguish informed and uninformed search.

Informed search uses problem-specific knowledge (a heuristic) to choose nodes; uninformed search uses only the problem definition.

Q3. Write the evaluation functions of greedy best-first search and A*.

Greedy: f(n) = h(n). A*: f(n) = g(n) + h(n).

Q4. What is an admissible heuristic?

One that never overestimates the true cost from a node to the goal: $h(n) \le h^*(n)$.

Q5. What is a consistent heuristic?

One for which $h(n) \le c(n, n') + h(n')$ for every successor n' of n.

Q6. Why is straight-line distance admissible for road routes?

No road between two places can be shorter than the straight line between them.

Q7. When does A* behave like uniform-cost search?

When $h(n) = 0$ for every node.

Q8. Give two heuristics for the 8-puzzle.

The number of misplaced tiles (h_1) and the sum of the tiles' Manhattan distances (h_2).

Q9. What is the main drawback of A*?

It keeps all generated nodes in memory.

Q10. Name two problems of hill climbing.

Any two of: local maxima, plateaus, ridges.

Long-answer questions (5–10 marks) — answer outlines

Q1. Explain best-first search and show how uniform-cost search, greedy best-first search and A* are special cases.

Heuristic h(n); g(n); evaluation function f(n); priority queue; Figure 1; the three f(n) formulas.

Q2. Explain greedy best-first search with an example and state its properties.

Figure 2; worked example 1 trace; not optimal (43 km vs 33 km); completeness; O(b^m).

Q3. Write the A* algorithm and trace it on a suitable example.

OPEN and CLOSED steps; goal test on removal; worked example 2 trace; comparison with uniform-cost search.

Q4. Explain admissible and consistent heuristics and show why A* with an admissible heuristic is optimal. What happens if h overestimates?

Definitions; h vs h* table; argument with a node on the cheapest route; consistency and monotone f; worked example 3.

Q5. Compare two heuristics for the 8-puzzle using dominance and the effective branching factor.

h_1 and h_2 for Figure 4; admissibility; table of nodes expanded and b*; relaxed problems.

Q6. Apply heuristic search to a field-robot path problem, interpret the results and discuss variants of A*.

Program, output and Figure 3; cost vs cells expanded; weighted A*, IDA*; uses in agriculture.

11. Glossary

Term	Meaning
A* search	Best-first search with f(n) = g(n) + h(n)
Admissible heuristic	Never overestimates the true cost to the goal
AO*	Heuristic search for AND–OR graphs (problem reduction)
Beam search	Keeps only the best k frontier nodes
Consistent (monotone) heuristic	h(n) ≤ c(n, n′) + h(n′) for every edge
Dominance	h_2 dominates h_1 if $h_2(n) \geq h_1(n)$ everywhere and both are admissible
Effective branching factor (b*)	Branching factor of a uniform tree holding the same number of nodes
Evaluation function f(n)	Value used by best-first search to rank frontier nodes
g(n)	Cost of the path from the start to node n
Greedy best-first search	Best-first search with f(n) = h(n)
Heuristic function h(n)	Estimated cost from node n to the goal
Hill climbing	Local search that moves to the best neighbouring state
IDA*	Iterative-deepening A*: depth-first search with an f-cost limit
Manhattan distance	Sum of row and column differences on a grid
Relaxed problem	A problem with some rules removed, used to derive heuristics
Weighted A*	A* with f(n) = g(n) + w·h(n), w > 1

References

Dechter, R., & Pearl, J. (1985). Generalized best-first search strategies and the optimality of A*. *Journal of the ACM, 32*(3), 505–536. https://doi.org/10.1145/3828.3830

Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics, 4*(2), 100–107. https://doi.org/10.1109/TSSC.1968.300136

Korf, R. E. (1985). Depth-first iterative-deepening: An optimal admissible tree search. *Artificial Intelligence, 27*(1), 97–109. https://doi.org/10.1016/0004-3702(85)90084-0

Pearl, J. (1984). *Heuristics: Intelligent search strategies for computer problem solving*. Addison-Wesley. ISBN 0-201-05594-5

Pohl, I. (1970). Heuristic search viewed as path finding in a graph. *Artificial Intelligence, 1*(3–4), 193–204. https://doi.org/10.1016/0004-3702(70)90007-X

Russell, S., & Norvig, P. (2020). *Artificial intelligence: A modern approach* (4th ed.). Pearson. ISBN 978-0-13-461099-3

Santos, L. C., Santos, F. N., Solteiro Pires, E. J., Valente, A., Costa, P., & Magalhães, S. (2020). Path planning for ground robots in agriculture: A short review. In *2020 IEEE International Conference on Autonomous Robot Systems and Competitions (ICARSC)* (pp. 61–66). IEEE. https://doi.org/10.1109/ICARSC49921.2020.9096177

All diagrams, examples and programs are original. The road map, field grid and 8-puzzle state are illustrative; every trace, count and result shown was checked by running the algorithms.

Practice MCQs

Q1. A heuristic function h(n) estimates

(a) the cost already spent to reach n
(b) the cost of the cheapest path from n to a goal
(c) the depth of n
(d) the number of children of n

Q2. Greedy best-first search expands the node with the lowest

(a) g(n)
(b) h(n)
(c) g(n) + h(n)
(d) depth

Q3. A* search ranks frontier nodes by

(a) f(n) = g(n)
(b) f(n) = h(n)
(c) f(n) = g(n) + h(n)
(d) f(n) = g(n) − h(n)

Q4. A heuristic is admissible if it

(a) never overestimates the true cost to the goal
(b) always overestimates the cost
(c) is always zero
(d) is computed by breadth-first search

Q5. A heuristic is consistent if, for every edge from n to n′,

(a) $h(n) \le c(n, n') + h(n')$
(b) $h(n) \ge c(n, n') + h(n')$
(c) $h(n) = 0$
(d) $g(n) = h(n)$

Q6. A* with h(n) = 0 for every node behaves like

(a) depth-first search
(b) greedy best-first search
(c) uniform-cost search
(d) hill climbing

Q7. The A* algorithm was introduced in 1968 by

(a) Dijkstra
(b) Hart, Nilsson and Raphael
(c) Turing
(d) Korf

Q8. On the farm-to-mandi map, greedy best-first search found a 43 km route while A* found

(a) 43 km
(b) 33 km
(c) 24 km
(d) 51 km

Q9. Straight-line distance is admissible for road routing because

(a) roads are always straight
(b) no road between two places is shorter than the straight line
(c) GPS is always accurate
(d) it is always zero

Q10. For the 8-puzzle, the Manhattan-distance heuristic h_2 compared with the misplaced-tiles heuristic h_1

(a) is inadmissible
(b) dominates h_1 and expands fewer nodes
(c) expands more nodes
(d) is identical to h_1

Q11. The main practical drawback of A* is that it

(a) is never optimal
(b) keeps all generated nodes in memory
(c) cannot use heuristics
(d) works only on trees

Q12. Hill climbing can fail to find the best solution because of

(a) local maxima, plateaus and ridges
(b) too much memory use
(c) admissible heuristics
(d) FIFO queues

Answers

Q1 — (b) the cost of the cheapest path from n to a goal
Q2 — (b) h(n)
Q3 — (c) f(n) = g(n) + h(n)

Q4 — (a) never overestimates the true cost to the goal

Q5 — (a) $h(n) \le c(n, n') + h(n')$

Q6 — (c) uniform-cost search

Q7 — (b) Hart, Nilsson and Raphael

Q8 — (b) 33 km

Q9 — (b) no road between two places is shorter than the straight line

Q10 — (b) dominates h_1 and expands fewer nodes

Q11 — (b) keeps all generated nodes in memory

Q12 — (a) local maxima, plateaus and ridges